

COM661 — Back-End API Development

Complete Code Listing

Module	COM661 — Back-End Web Development
Application	Carbon Footprint Tracking API
Framework	Flask (Python 3.9)
Database	MongoDB — carbonDB
Authentication	Basic Auth (login) + JWT Bearer Token
Date	11 March 2026

1. app.py — Application Entry Point

```
from flask import Flask
from flask_pymongo import PyMongo
from flask_jwt_extended import JWTManager
from flask_cors import CORS
from datetime import timedelta
from routes.auth_routes import auth_bp, init_auth_routes
from routes.activity_routes import activity_bp, init_activity_routes
from routes.analytics_routes import analytics_bp, init_analytics_routes
app = Flask(__name__)
app.config["MONGO_URI"] = "mongodb://localhost:27017/carbonDB"
app.config["JWT_SECRET_KEY"] = "supersecretkey123"
app.config["JWT_ACCESS_TOKEN_EXPIRES"] = timedelta(days=30)
mongo = PyMongo(app)
jwt = JWTManager(app)
CORS(app)
init_auth_routes(mongo)
init_activity_routes(mongo)
init_analytics_routes(mongo)
app.register_blueprint(auth_bp)
app.register_blueprint(activity_bp)
app.register_blueprint(analytics_bp)
if __name__ == "__main__":
    app.run(debug=True, port=5001)
```

2. routes/auth_routes.py — Authentication & User Management

```

from flask import Blueprint, request, jsonify
import bcrypt
from flask_jwt_extended import (create_access_token, create_refresh_token,
jwt_required, get_jwt_identity, get_jwt)
from flask_httpauth import HTTPBasicAuth
auth_bp = Blueprint('auth', __name__)
mongo = None
basic_auth = HTTPBasicAuth()
def init_auth_routes(mongo_instance):
    global mongo
    mongo = mongo_instance
    @basic_auth.verify_password
    def verify_password(email, password):
        user = mongo.db.users.find_one({"email": email})
        if user and bcrypt.checkpw(password.encode('utf-8'),
        user['password'].encode('utf-8')):
            return user
        return None
    # REGISTER
    @auth_bp.route('/users/register', methods=['POST'])
    def register():
        data = request.get_json()
        if not data.get('email') or not data.get('password') or not data.get('name'):
            return jsonify({"error": "Name, email and password are required"}), 400
        if mongo.db.users.find_one({"email": data['email']}):
            return jsonify({"error": "Email already registered"}), 400
        hashed_pw = bcrypt.hashpw(data['password'].encode('utf-8'), bcrypt.gensalt())
        new_user = {
            "_id": "USR-" + data['name'][:3].upper(),
            "name": data['name'],
            "email": data['email'],
            "password": hashed_pw.decode('utf-8'),
            "role": data.get('role', 'user')
        }
        mongo.db.users.insert_one(new_user)
        return jsonify({"message": "User registered successfully"}), 201
    # LOGIN (Basic Auth) - returns JWT token
    @auth_bp.route('/users/login', methods=['POST'])
    @basic_auth.login_required
    def login():
        user = basic_auth.current_user()
        token = create_access_token(
            identity=user['_id'],
            additional_claims={"role": user['role']}
        )
        refresh_token = create_refresh_token(identity=user['_id'])
        return jsonify({
            "token": token,
            "refresh_token": refresh_token,
            "role": user['role']
        }), 200
    # REFRESH TOKEN
    @auth_bp.route('/users/refresh', methods=['POST'])
    @jwt_required(refresh=True)
    def refresh():
        current_user = get_jwt_identity()
        user = mongo.db.users.find_one({"_id": current_user})

```

```

new_token = create_access_token(
    identity=current_user,
    additional_claims={"role": user['role']}
)
return jsonify({"token": new_token}), 200
# GET ALL USERS (admin only)
@auth_bp.route('/users', methods=['GET'])
@jwt_required()
def get_all_users():
    claims = get_jwt()
    if claims.get('role') != 'admin':
        return jsonify({"error": "Admin access required"}), 403
    users = list(mongo.db.users.find({}, {"_id": 0, "password": 0}))
    return jsonify({"data": users, "total": len(users)}), 200
# GET ONE USER
@auth_bp.route('/users/<user_id>', methods=['GET'])
@jwt_required()
def get_user(user_id):
    current_user = get_jwt_identity()
    claims = get_jwt()
    if current_user != user_id and claims.get('role') != 'admin':
        return jsonify({"error": "Unauthorised"}), 403
    user = mongo.db.users.find_one({"_id": user_id}, {"_id": 0, "password": 0})
    if not user:
        return jsonify({"error": "User not found"}), 404
    return jsonify({"data": user}), 200
# UPDATE USER
@auth_bp.route('/users/<user_id>', methods=['PUT'])
@jwt_required()
def update_user(user_id):
    current_user = get_jwt_identity()
    claims = get_jwt()
    data = request.get_json()
    if current_user != user_id and claims.get('role') != 'admin':
        return jsonify({"error": "Unauthorised"}), 403
    user = mongo.db.users.find_one({"_id": user_id})
    if not user:
        return jsonify({"error": "User not found"}), 404
    data.pop('password', None); data.pop('role', None); data.pop('_id', None)
    mongo.db.users.update_one({"_id": user_id}, {"$set": data})
    return jsonify({"message": "User updated successfully"}), 200
# DELETE USER
@auth_bp.route('/users/<user_id>', methods=['DELETE'])
@jwt_required()
def delete_user(user_id):
    current_user = get_jwt_identity()
    claims = get_jwt()
    if current_user != user_id and claims.get('role') != 'admin':
        return jsonify({"error": "Unauthorised"}), 403
    user = mongo.db.users.find_one({"_id": user_id})
    if not user:
        return jsonify({"error": "User not found"}), 404
    mongo.db.users.delete_one({"_id": user_id})
    return jsonify({"message": "User deleted successfully"}), 200

```

3. routes/activity_routes.py — Activity CRUD, Sources & Filtering

```

from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required, get_jwt_identity, get_jwt
from utils.validators import validate_activity_data
from utils.emission_calculator import calculate_carbon_footprint
import uuid
from datetime import datetime

activity_bp = Blueprint('activities', __name__)
mongo = None

def init_activity_routes(mongo_instance):
    global mongo
    mongo = mongo_instance

    # CREATE
    @activity_bp.route('/activities', methods=['POST'])
    @jwt_required()
    def create_activity():
        data = request.get_json()
        current_user = get_jwt_identity()
        errors = validate_activity_data(data)
        if errors:
            return jsonify({"errors": errors}), 400
        calculated_emission = calculate_carbon_footprint(
            data['activity_value'], data['emission_factor'])
        new_activity = {
            "_id": "ACT-" + str(uuid.uuid4())[0:8].upper(),
            "user_id": current_user,
            "activity_type": data['activity_type'],
            "activity_detail": data.get('activity_detail', ''),
            "activity_value": data['activity_value'],
            "emission_factor": data['emission_factor'],
            "carbon_emission": calculated_emission,
            "unit": data['unit'],
            "timestamp": data.get('timestamp', datetime.utcnow().strftime('%Y-%m-%d')),
            "sources": []
        }
        mongo.db.activities.insert_one(new_activity)
        activity_id = new_activity['_id']
        new_activity.pop('_id')
        return jsonify({"message": "Activity logged successfully",
            "data": new_activity, "_id": activity_id}), 201

    # GET ALL
    @activity_bp.route('/activities', methods=['GET'])
    @jwt_required()
    def get_activities():
        current_user = get_jwt_identity()
        claims = get_jwt(); role = claims.get('role')
        if role == 'admin':
            activities = list(mongo.db.activities.find({}, {"_id": 0}))
        else:
            activities = list(mongo.db.activities.find(
                {"user_id": current_user}, {"_id": 0}))
        return jsonify({"data": activities, "total": len(activities)}), 200

    # GET ONE
    @activity_bp.route('/activities/<activity_id>', methods=['GET'])
    @jwt_required()
    def get_activity(activity_id):
        current_user = get_jwt_identity()
        claims = get_jwt(); role = claims.get('role')

```

```

activity = mongo.db.activities.find_one({"_id": activity_id}, {"_id": 0})
if not activity:
    return jsonify({"error": "Activity not found"}), 404
if role != 'admin' and activity['user_id'] != current_user:
    return jsonify({"error": "Unauthorised"}), 403
return jsonify({"data": activity}), 200

# UPDATE
@activity_bp.route('/activities/<activity_id>', methods=['PUT'])
@jwt_required()
def update_activity(activity_id):
    data = request.get_json()
    current_user = get_jwt_identity()
    claims = get_jwt(); role = claims.get('role')
    activity = mongo.db.activities.find_one({"_id": activity_id})
    if not activity:
        return jsonify({"error": "Activity not found"}), 404
    if role != 'admin' and activity['user_id'] != current_user:
        return jsonify({"error": "Unauthorised"}), 403
    new_value = data.get('activity_value', activity['activity_value'])
    new_factor = data.get('emission_factor', activity['emission_factor'])
    data['carbon_emission'] = calculate_carbon_footprint(new_value, new_factor)
    data.pop('_id', None); data.pop('user_id', None)
    mongo.db.activities.update_one({"_id": activity_id}, {"$set": data})
    return jsonify({"message": "Activity updated successfully"}), 200

# DELETE
@activity_bp.route('/activities/<activity_id>', methods=['DELETE'])
@jwt_required()
def delete_activity(activity_id):
    current_user = get_jwt_identity()
    claims = get_jwt(); role = claims.get('role')
    activity = mongo.db.activities.find_one({"_id": activity_id})
    if not activity:
        return jsonify({"error": "Activity not found"}), 404
    if role != 'admin' and activity['user_id'] != current_user:
        return jsonify({"error": "Unauthorised"}), 403
    mongo.db.activities.delete_one({"_id": activity_id})
    return jsonify({"message": "Activity deleted successfully"}), 200

# ADD SOURCE
@activity_bp.route('/activities/<activity_id>/sources', methods=['POST'])
@jwt_required()
def add_source(activity_id):
    data = request.get_json()
    activity = mongo.db.activities.find_one({"_id": activity_id})
    if not activity:
        return jsonify({"error": "Activity not found"}), 404
    new_source = {
        "source_id": "SRC-" + str(uuid.uuid4())[6].upper(),
        "description": data.get('description', ''),
        "distance_km": data.get('distance_km', 0),
        "emission_kg": data.get('emission_kg', 0)
    }
    mongo.db.activities.update_one(
        {"_id": activity_id}, {"$push": {"sources": new_source}})
    return jsonify({"message": "Source added successfully", "source": new_source}), 201

# UPDATE SOURCE
@activity_bp.route('/activities/<activity_id>/sources/<source_id>', methods=['PUT'])
@jwt_required()
def update_source(activity_id, source_id):
    data = request.get_json()

```

```

activity = mongo.db.activities.find_one({"_id": activity_id})
if not activity:
    return jsonify({"error": "Activity not found"}), 404
source = next((s for s in activity['sources'] if s['source_id'] == source_id), None)
if not source:
    return jsonify({"error": "Source not found"}), 404
update_fields = {}
if 'description' in data: update_fields["sources.$.description"] = data['description']
if 'distance_km' in data: update_fields["sources.$.distance_km"] = data['distance_km']
if 'emission_kg' in data: update_fields["sources.$.emission_kg"] = data['emission_kg']
mongo.db.activities.update_one(
    {"_id": activity_id, "sources.source_id": source_id}, {"$set": update_fields})
return jsonify({"message": "Source updated successfully"}), 200

# DELETE SOURCE
@activity_bp.route('/activities/<activity_id>/sources/<source_id>', methods=['DELETE'])
@jwt_required()
def delete_source(activity_id, source_id):
    activity = mongo.db.activities.find_one({"_id": activity_id})
    if not activity:
        return jsonify({"error": "Activity not found"}), 404
    source = next((s for s in activity['sources'] if s['source_id'] == source_id), None)
    if not source:
        return jsonify({"error": "Source not found"}), 404
    mongo.db.activities.update_one(
        {"_id": activity_id}, {"$pull": {"sources": {"source_id": source_id}}})
    return jsonify({"message": "Source deleted successfully"}), 200

# FILTER BY TYPE
@activity_bp.route('/activities/filter/type', methods=['GET'])
@jwt_required()
def filter_by_type():
    current_user = get_jwt_identity()
    claims = get_jwt(); role = claims.get('role')
    activity_type = request.args.get('activity_type')
    if not activity_type:
        return jsonify({"error": "activity_type parameter is required"}), 400
    query = {"activity_type": activity_type}
    if role != 'admin': query["user_id"] = current_user
    activities = list(mongo.db.activities.find(query, {"_id": 0}))
    return jsonify({"data": activities, "total": len(activities)}), 200

# FILTER BY DATE
@activity_bp.route('/activities/filter/date', methods=['GET'])
@jwt_required()
def filter_by_date():
    current_user = get_jwt_identity()
    claims = get_jwt(); role = claims.get('role')
    start_date = request.args.get('start_date')
    end_date = request.args.get('end_date')
    if not start_date or not end_date:
        return jsonify({"error": "start_date and end_date are required"}), 400
    query = {"timestamp": {"$gte": start_date, "$lte": end_date}}
    if role != 'admin': query["user_id"] = current_user
    activities = list(mongo.db.activities.find(query, {"_id": 0}))
    return jsonify({"data": activities, "total": len(activities)}), 200

# FILTER BY EMISSION
@activity_bp.route('/activities/filter/emission', methods=['GET'])
@jwt_required()
def filter_by_emission():
    current_user = get_jwt_identity()
    claims = get_jwt(); role = claims.get('role')

```

```
min_emission = request.args.get('min_emission')
if not min_emission:
    return jsonify({"error": "min_emission parameter is required"}), 400
query = {"carbon_emission": {"$gte": float(min_emission)}}
if role != 'admin': query["user_id"] = current_user
activities = list(mongo.db.activities.find(query, {"_id": 0}))
return jsonify({"data": activities, "total": len(activities)}), 200

# TOP EMITTING
@activity_bp.route('/activities/top-emitting', methods=['GET'])
@jwt_required()
def top_emitting():
    current_user = get_jwt_identity()
    claims = get_jwt(); role = claims.get('role')
    query = {} if role == 'admin' else {"user_id": current_user}
    activities = list(mongo.db.activities.find(
        query, {"_id": 0}).sort("carbon_emission", -1))
    return jsonify({"data": activities, "total": len(activities)}), 200
```


4. routes/analytics_routes.py — MongoDB Aggregation Analytics

```

from flask import Blueprint, jsonify
from flask_jwt_extended import jwt_required, get_jwt_identity
analytics_bp = Blueprint('analytics', __name__)

mongo = None

def init_analytics_routes(mongo_instance):
    global mongo
    mongo = mongo_instance

    # TOTAL EMISSIONS
    @analytics_bp.route('/analytics/total-emissions', methods=['GET'])
    @jwt_required()
    def total_emissions():
        current_user = get_jwt_identity()
        pipeline = [
            {"$match": {"user_id": current_user}},
            {"$group": {
                "_id": "$user_id",
                "total_emission_kg": {"$sum": "$carbon_emission"},
                "total_activities": {"$sum": 1}
            }}
        ]
        result = list(mongo.db.activities.aggregate(pipeline))
        return jsonify({"data": result}), 200

    # EMISSIONS BY TYPE
    @analytics_bp.route('/analytics/emissions-by-type', methods=['GET'])
    @jwt_required()
    def emissions_by_type():
        current_user = get_jwt_identity()
        pipeline = [
            {"$match": {"user_id": current_user}},
            {"$group": {
                "_id": "$activity_type",
                "total_emission_kg": {"$sum": "$carbon_emission"},
                "count": {"$sum": 1}
            }},
            {"$sort": {"total_emission_kg": -1}}
        ]
        result = list(mongo.db.activities.aggregate(pipeline))
        return jsonify({"data": result}), 200

    # MONTHLY TRENDS
    @analytics_bp.route('/analytics/monthly-trends', methods=['GET'])
    @jwt_required()
    def monthly_trends():
        current_user = get_jwt_identity()
        pipeline = [
            {"$match": {"user_id": current_user}},
            {"$group": {
                "_id": {
                    "year": {"$substr": ["$timestamp", 0, 4]},
                    "month": {"$substr": ["$timestamp", 5, 2]}
                },
                "total_emission_kg": {"$sum": "$carbon_emission"},
                "activity_count": {"$sum": 1}
            }},
            {"$sort": {"_id.year": 1, "_id.month": 1}}
        ]
        result = list(mongo.db.activities.aggregate(pipeline))
        return jsonify({"data": result}), 200

```

```
# HIGHEST ACTIVITIES
@analytics_bp.route('/analytics/highest-activities', methods=['GET'])
@jwt_required()
def highest_activities():
    current_user = get_jwt_identity()
    pipeline = [
        {"$match": {"user_id": current_user}},
        {"$group": {
            "_id": "$activity_type",
            "total_emission_kg": {"$sum": "$carbon_emission"},
            "avg_emission_kg": {"$avg": "$carbon_emission"},
            "count": {"$sum": 1}
        }},
        {"$sort": {"total_emission_kg": -1}}
    ]
    result = list(mongo.db.activities.aggregate(pipeline))
    return jsonify({"data": result}), 200
```

5. utils/validators.py — Input Validation

```
def validate_activity_data(data):
    """
    Validates activity input and ensures numeric integrity.
    """
    errors = []
    # Check for required top-level fields
    required_fields = ['activity_type', 'activity_value', 'emission_factor', 'unit']
    for field in required_fields:
        if field not in data:
            errors.append(f"Missing required field: {field}")
    # Type and Range Checking
    try:
        if float(data.get('activity_value', 0)) <= 0:
            errors.append("activity_value must be a positive number.")
        if float(data.get('emission_factor', 0)) < 0:
            errors.append("emission_factor cannot be negative.")
    except (ValueError, TypeError):
        errors.append("activity_value and emission_factor must be numeric.")
    # Validate activity_type against allowed categories
    allowed_types = ['Transport', 'Energy', 'Consumption']
    if data.get('activity_type') not in allowed_types:
        errors.append(
            f"Invalid activity_type. Must be one of: {' '.join(allowed_types)}")
    return errors
```

6. utils/emission_calculator.py — Carbon Footprint Calculator

```
def calculate_carbon_footprint(value, factor):  
    try:  
        # Automatic calculation to ensure data integrity  
        return round(float(value) * float(factor), 2)  
    except (ValueError, TypeError):  
        return 0.0
```